

Git 과 협업 워크플로우

: Level 1 · 4강

HANT



이 시간의 목표

커밋 · 브랜치를 스냅샷과 포인터로 설명하고 4개 공간의 이동 명령을 사용한다

merge 와 rebase 의 차이를 이력 관점에서 비교하고 상황에 맞게 선택한다

충돌을 해결하고 PR 기반 협업 사이클을 수행한다

ROS2 생태계의 모든 패키지가 GitHub 위에서 개발됩니다.
Git 은 도구 하나가 아니라 이 분야에서 일하는 방식입니다.

목차



순서	주제	예상 소요 시간
1부	버전 관리의 필요와 Git 의 구조	25분
2부	브랜치 · merge · rebase · 충돌	40분
3부	Pull Request 와 코드 리뷰, CI	25분

1부

Git 은 무엇을 저장할까요

팀원 둘이 같은 파일을 고쳤습니다

① 로봇 프로젝트의 흔한 사고

버전 관리가 없을 때

덮어쓴 순간 하루치가
사라집니다
되돌릴 방법이 없습니다

“지난주엔 잘 돌아갔는데”

Git 이 있을 때

모든 시점이 스냅샷으로 남고
각자의 변경이 따로 보관됩니다
언제든 그 시점으로 갑니다

“누가 무엇을 왜 바꿨다”

↳ 버전 관리는 백업이 아니라 여러 사람이 동시에 일하기 위한 구조입니다.

먼저 두 단어만 잡겠습니다

Git 을 이해한다는 것은 결국 “커밋 노드의 그래프와 그것을 가리키는 포인터”를 이해하는 일입니다.

스냅샷

차이가 아닙니다

그 시점 프로젝트 전체의 모습을 통째로 저장합니다

커밋

스냅샷 하나입니다

고유 해시 a3f9c2e 를 갖고 부모를 가리킵니다

이력

커밋의 사슬입니다

부모를 따라 거슬러 올라가면 과거 전체가 나옵니다

분산(distributed) — 서버가 죽어도 이력은 남습니다

복사본

모든 개발자의 PC 에 저장소 전체 이력의 완전한 복사본이 있습니다
GitHub 는 "중앙"이 아니라 팀이 합의한 만남의 장소일 뿐입니다

오프라인

커밋 · 이력 조회 · 브랜치 생성이 네트워크 없이 전부 됩니다
네트워크가 필요한 것은 push · pull · clone 세 가지뿐입니다

로봇 현장

공장 · 야외처럼 네트워크가 불안정한 곳에서 특히 고마운 특성입니다
로봇 위에서 직접 고치고 커밋한 뒤, 돌아와서 push 하면 됩니다

이 강의의 포인트

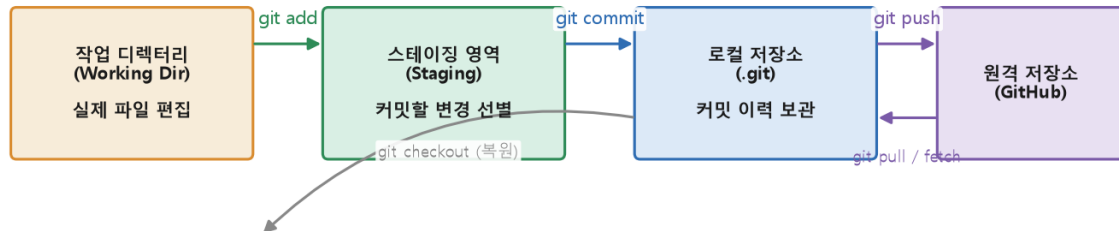
SVN 같은 중앙집중형과 갈라지는 지점입니다

중앙집중형은 서버에 못 붙으면 커밋조차 못 합니다.

Git 은 로컬에 이력 전체가 있으므로 혼자서도 완전한 버전 관리가 됩니다.

내 변경이 지금 어디에 있는지 — 공간을 넷으로 나눅니다

git의 4개 공간과 데이터의 이동



commit = 스냅샷 저장, push/pull = 원격과 동기화

해석

작업 디렉터리 → (add) → 스테이징 → (commit) → 로컬 저장소 → (push) → 원격

스테이징이 중간에 있는 이유는 수정한 것 중 일부만 골라 담기 위해서입니다.

“의미 있는 단위로 커밋”이 가능해지는 자리입니다.

매일 반복하는 다섯 줄

확인

git status — 어떤 파일이 바뀌었는지. 무엇을 하기 전에도 먼저 칩니다
git diff — 구체적으로 무엇이 바뀌었는지 줄 단위로 봅니다

담기 · 기록

git add src/lidar_driver.py — 커밋할 변경만 골라 스테이징에
git commit -m "LiDAR 타임아웃 5초로 연장" — 스냅샷으로 기록

공유

git push origin main — 원격에 올려 팀과 공유합니다
git pull — 팀의 변경을 받아옵니다. 작업 시작 전에 먼저 당깁니다

이 강의의 포인트

status 와 **diff** 를 치는 습관이 사고의 절반을 막습니다

무엇이 스테이징에 있는지 모른 채 커밋하면 엉뚱한 파일이 함께 들어갑니다.
git log --oneline --graph 로 이력을 그래프로 보는 것도 같은 습관입니다.

커밋 메시지는 미래의 팀원에게 보내는 기록입니다.

무엇을, 왜 — 두 가지가 담기게

- 나쁜 예 — “수정”, “fix”, “ㅇㅇ”, “작업중”. 6개월 뒤 아무 정보가 없습니다
- 좋은 예 — “LiDAR 드라이버 타임아웃 5초로 연장 (야외 저조도 재시도 대응)”
- 한 커밋 = 한 가지 일 — 리팩터링과 기능 추가를 한 커밋에 섞지 않습니다
- 되돌릴 수 있는 단위 로 자릅니다. 커밋이 크면 문제가 생겨도 그 부분만 뺄 수 없습니다
- 6개월 뒤 “왜 이렇게 했지” 를 묻는 사람은 **대개 자기 자신입니다**

2부

갈라지고 합쳐지는 법

브랜치는 무겁지 않습니다. 포인터 하나일 뿐입니다.



브랜치 — 서로 방해하지 않는 평행 우주

- **정체** — 브랜치는 특정 커밋을 가리키는 41바이트짜리 포인터입니다
- **그래서** 만드는 데 비용이 사실상 없습니다. 부담 없이 만들고 지웁니다
- **git switch -c feature/lidar-driver** — 생성과 동시에 이동
- **git switch main** — 언제든지 복귀. 작업물은 브랜치 쪽에 그대로 남아 있습니다
- 커밋하면 포인터가 새 커밋으로 따라 옮겨 갑니다. **갈라짐은 그렇게 생깁니다**

규칙은 한 줄입니다 — main 은 항상 동작하는 코드

main	지금 로봇에 올려도 되는 코드만 둡니다. 검증을 통과한 것만 들어웁니다 직접 커밋하지 않습니다. 병합으로만 갱신됩니다
작업 브랜치	실험적인 제어 게인 튜닝, 새 센서 드라이버는 전부 feature/... 에서 feature/lidar-driver · fix/odom-drift 처럼 목적이 보이는 이름으로
수명	작게 · 짧게 — 하루에서 며칠 단위로 끝내고 병합합니다 오래 살아 있는 브랜치는 반드시 큰 충돌로 돌아옵니다

이 강의의 포인트

브랜치 수명이 짧을수록 충돌이 작아집니다

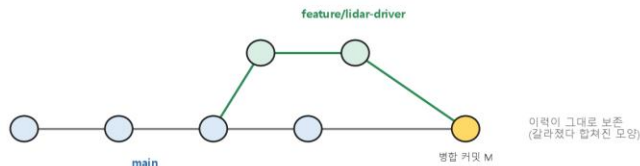
두 주 동안 혼자 굴린 브랜치는 병합 시점에 수십 개의 충돌을 냅니다.

자주 잘라 자주 합치는 것이 결국 가장 빠릅니다.

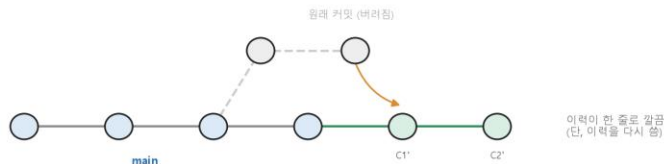
합치는 방법이 둘입니다 — 결과 코드는 같고, 이력이 다릅니다

merge vs rebase — 같은 목표, 다른 이력

merge: 두 이력을 합치는 '병합 커밋'을 만든다



rebase: 내 커밋을 최신 main 위로 ' 옮겨 다시 쌓는다'



해석

(위) merge 는 병합 커밋 M 을 만들고, (아래) rebase 는 커밋을 옮겨 다시 쌓습니다

merge 는 “언제 갈라졌고 언제 합쳐졌는지”를 사실 그대로 보존합니다.

rebase 는 원래 커밋을 버리고 C1' · C2' 라는 새 커밋으로 다시 씁니다.

명령으로 보면

merge 방식	git switch main git merge feature/lidar-driver → 병합 커밋 M 이 생깁니다
rebase 방식	git switch feature/lidar-driver ; git rebase main git switch main ; git merge ... → 한 줄이라 fast-forward 로 붙습니다
무엇이 다른가	merge — 이력에 갈라짐이 남습니다. 사실의 기록입니다 rebase — 이력이 한 줄이 됩니다. 읽기 좋게 편집한 기록입니다

이 강의의 포인트

정답은 없고, 팀의 합의가 있습니다

“이력은 사실이어야 한다” 쪽과 “이력은 읽히기 위한 것이다” 쪽의 오래된 논쟁입니다.

실무는 대개 둘을 자리별로 나눠 씁니다 — 다음 장.

상황에 따라 무엇을 쓸까요

CASE 01

기능을 공유 main 에 합치기

merge

PR 병합 · 이력 보존

CASE 02

내 브랜치를 최신 main 과 동기화

rebase

중간 병합 커밋 없이

CASE 03

이미 push 한 공유 커밋

rebase 금지

팀 이력과 충돌

CASE 04

커밋 메시지를 다듬고 싶다

로컬만 OK

push 전까지만

CASE 05

병합 이력이 지저분해 보인다

그대로 둡니다

사실이 더 중요

push 한 커밋은 rebase 하지 않습니다.



이 강의에서 하나만 가져간다면

- **이유** — rebase 는 커밋을 옮기는 것이 아니라 새 해시로 다시 쓰는 작업입니다
- **결과** — 팀원의 로컬에는 옛 커밋이, 원격에는 새 커밋이 남아 이력이 갈라집니다
- **증상** — 강제 push 로 남의 커밋이 사라지거나, pull 마다 유령 충돌이 납니다
- **안전 지대** — 아직 나만 가진 로컬 커밋. 여기서는 rebase 를 마음껏 씁니다
- 판단 기준은 하나입니다 — **다른 사람이 이 커밋을 봤을 수 있는가**

충돌은 오류가 아닙니다

다른 파일 · 다른 줄을 고쳤다면 Git 은 아무 말 없이 알아서 합칩니다. 충돌은 생각보다 드물게 납니다.

언제

같은 줄을 다르게

같은 파일의 같은 부분을 양쪽에서 고쳤을 때만 납니다

왜

Git 이 못 정합니다

어느 쪽이 옳은지는 코드의 의도를 아는 사람만 압니다

본질

사람에게 묻는 것

“여기는 정해 주세요”라는 정상적인 질문입니다

파일 안에 표식이 삽입됩니다.



get_max_speed() 에서 충돌이 났다면

- <<<<<<< HEAD ← 여기서부터 지금 있는 쪽(main)의 내용
- return 0.8 # 안전을 위해 감속 — main 의 변경
- ===== ← 경계선
- return 1.2 # 성능 테스트용 증속 — 내 브랜치의 변경
- >>>>>> feature/speed-tuning ← 가져오려는 쪽의 끝

해결은 세 단계입니다 "둘 중 하나를 고르는 것이 아닙니다.
두 변경의 의도를 읽고,
올바른 최종 코드를 새로 쓰는 일입니다."

Step 01

직접 고쳐 씁니다

두 버전을 읽고 최종 코드를 작성합니다.
<<<<<< ===== >>>>>> 는 모두 지웁니다.

Step 02

해결했음을 표시

git add 파일명
— 스테이징이 곧 "해결 완료" 신호입니다.

Step 03

마무리

git commit (merge 중)
git rebase --continue (rebase 중)

충돌 해결은 기계적 선택이 아닙니다

제3의 코드

0.8 이냐 1.2 냐가 아니라 “안전 한도 0.8 유지 + 테스트 플래그로 1.2 허용”
두 변경의 의도를 모두 살리는 코드가 정답인 경우가 많습니다

확인

해결 뒤에는 반드시 빌드하고 테스트를 돌립니다
표식만 지워 컴파일은 되는데 로직이 깨진 상태가 가장 위험합니다

예방

브랜치를 작게 · 짧게 유지합니다 (하루~며칠)
main 의 변경을 자주 가져와 격차를 줄입니다 — git pull 후 rebase

이 강의의 포인트

충돌의 크기는 브랜치의 나이에 비례합니다

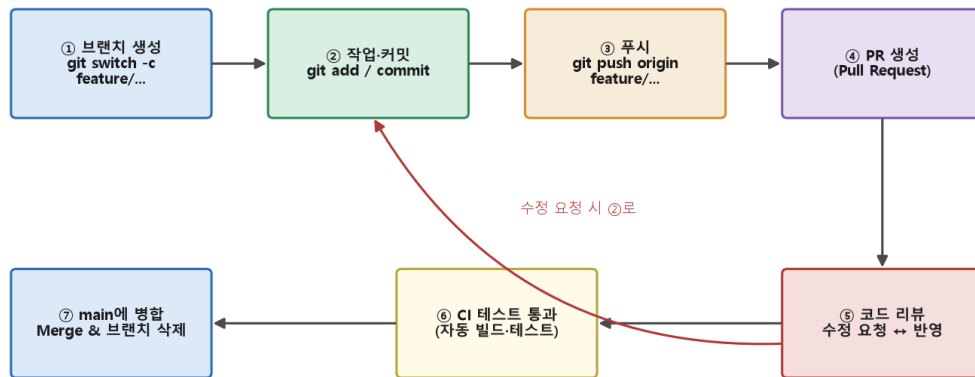
두 사람이 같은 파일을 오래 따로 고치면 충돌은 필연입니다.
자주 합치면 매번 작게, 미루면 한 번에 크게 옵니다.

3부

팀으로 일하는 법

브랜치가 main 에 들어가는 관문 — Pull Request

Pull Request 협업 사이클



main 브랜치는 항상 '동작하는 코드'로 보호되고, 모든 변경은 리뷰와 테스트를 거쳐 들어갑니다

해석

① 브랜치 → ② 커밋 → ③ 푸시 → ④ PR → ⑤ 리뷰 → ⑥ CI → ⑦ 병합·브랜치 삭제

⑤ 에서 수정 요청이 오면 ② 로 돌아가 커밋을 추가합니다 — PR 은 살아 있는 대화입니다.

main 은 리뷰와 테스트를 통과한 코드만 받는 보호 구역이 됩니다.

코드 리뷰는 형식적 절차가 아닙니다

리뷰에 쓰는 30분이 현장에서 로봇을 세우는 3일을 막습니다.



결함 조기 발견 — 로봇 코드의 버그는 물리적 사고가 됩니다. 두 번째 눈이 결함 검출률을 크게 올립니다



지식 공유 — 팀 전체가 코드베이스를 이해하게 되고, “그 부분은 A만 안다”는 위험이 줄어듭니다



컨벤션 유지 — 코드 스타일과 설계 원칙이 문서가 아니라 대화를 통해 전파됩니다



기록 — “왜 이 설계를 골랐는가”가 PR 스레드에 남습니다. 커밋 메시지가 못 담은 맥락입니다

리뷰는 사람이 아니라 코드를 봅니다.



리뷰어와 작성자, 각자의 태도

- 리뷰어 — 코드가 아니라 코드의 문제를 지적합니다. 사람을 향한 말은 쓰지 않습니다
- 작성자 — 방어하지 말고 의도를 설명합니다. 설명이 길어지면 코드가 불친절한 것입니다
- PR은 작게 — 수백 줄 이내. 1000줄짜리 PR은 아무도 제대로 못 읽고 "LGTM"만 달립니다
- 질문 형태로 — "이 경우 타임아웃은 어떻게 되나요?"가 "이건 틀렸어요"보다 잘 통합니다
- 서로를 돕는 가장 확실한 방법은 리뷰 가능한 크기로 자르는 것입니다

심화 — CI(지속적 통합)와 로봇 프로젝트

무엇

푸시 · PR 마다 서버가 자동으로 빌드와 테스트를 돌리는 체계입니다
GitHub Actions 등이 PR 화면에 통과 · 실패를 초록/빨강으로 붙입니다

무엇을 돌리나

colcon build 가 깨지지 않는지 (컴파일 검증)
단위 테스트(pytest · gtest — 모듈 ②) 통과 여부, 코드 스타일 검사

하드웨어

실기가 필요한 테스트는 시뮬레이션(Gazebo)으로 대체합니다
야간에 실기 테스트 팜에서 돌리는 팀도 있습니다

이 강의의 포인트

핵심 효과는 “main 이 깨진 채로 하루가 시작되는 일”의 차단입니다

사람이 잊어도 CI 는 잊지 않습니다. 규율을 자동화로 바꾸는 장치입니다.

PR 사이클 ⑥ 이 바로 이 자리입니다.

심화 — 되돌리는 도구는 대상 공간마다 다릅니다

CASE 01

작업 디렉터리의 수정을 버린다

restore

git restore 파일

CASE 02

add 한 것을 스테이징에서 뺀다

restore

--staged 파일

CASE 03

공유된 잘못된 커밋을 무효화

revert

반대 변경을 추가

CASE 04

나만 가진 최근 커밋을 물린다

reset

--soft HEAD~1

CASE 05

커밋을 잃은 것 같다

reflog

HEAD 이동 기록

원칙은 rebase 때와 같습니다

Git 에서 위험한 명령의 목록은 짧습니다 — 공유된 이력을 다시 쓰는 것들뿐입니다.

공유된 이력

추가로 되돌립니다

revert — 반대 변경을 새 커밋으로 쌓습니다. 이력이 보존됩니다

로컬 이력

수정으로 되돌립니다

reset — 포인터를 뒤로 옮깁니다. 아직 나만 가진 커밋에만

판단

누가 봤는가

이 커밋을 다른 사람이 봤다면 무조건 revert 입니다

오늘 정리



이것만 남기면 됩니다

- 구조 — 커밋은 스냅샷 노드, 브랜치는 그것을 가리키는 포인터입니다
- 이동 — 작업 디렉터리 → add → 스테이징 → commit → 로컬 → push → 원격
- 합치기 — merge 는 이력 보존, rebase 는 이력 정리. 공유된 커밋은 rebase 금지
- 충돌 — 오류가 아니라 질문입니다. 두 변경의 의도를 통합해 새로 씁니다
- 협업 — 브랜치 → 커밋 → 푸시 → PR → 리뷰 → CI → 병합이 **팀의 표준 사이클입니다**

미니 퀴즈

(각 2점 · 총 10점)

스스로 확인하기

- 1. Git 의 4개 공간 중 git add 로 파일이 이동하는 곳은?

- 2. 이미 push 한 커밋에 rebase 를 쓰면 안 되는 이유를 설명하세요.

- 3. 공유 main 의 잘못된 커밋을 이력을 보존하며 되돌리는 명령은?

- 4. Pull Request 를 쓰는 이유를 두 가지 쓰세요.

- 5. .gitignore 에 넣어야 할 것과 넣으면 안 되는 것을 하나씩 쓰세요.

오늘의 실습

“명령을 따라 친 것으로 끝내지 마세요.

충돌 표식이 무엇을 뜻했는지 한 줄로 적는 것까지가
이 실습입니다.”

Step 01

기본 사이클

clone → 수정 → status-diff → add →
commit → push. GitHub 에서 확인합니다.

Step 02

브랜치와 PR

feature/readme-update 로 수정-푸시하고
PR 을 열어 스스로 리뷰 후 병합합니다.

Step 03

충돌과 rebase

branch-a-b 로 같은 줄을 다르게 고쳐
충돌을 내고 해결, 이력 모양을 비교합니다.

더 읽을 거리



참고 문헌

- **S. Chacon, B. Straub**, Pro Git, 2nd ed., Apress, 2014 — 무료 공개. 내부 구조까지 다루는 정본
- **Git Reference** (git-scm.com/docs) — 명령마다의 1차 자료입니다
- **Learn Git Branching** (learngitbranching.js.org) — merge · rebase 를 그래프로 체험합니다
- **GitHub Docs** — Pull requests · Actions. PR 과 CI 설정의 표준 안내서
- **ROS 2 Contributing Guide** — 실제 로봇 오픈소스가 PR 을 어떻게 다루는지 봅니다

Q&A

무엇이든 물어보세요

소프트웨어는 결국 팀이 만듭니다.

다음 시간부터는 모듈 ② — C++ · Python · ROS2